

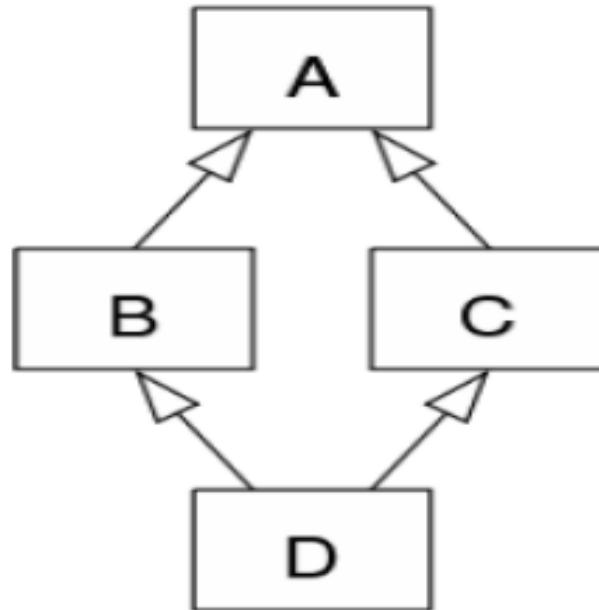
Week 9 lec 3

Week 10 lec 1

Diamond problem

- In case of hybrid inheritance, a Diamond problem may arise. The “dreaded diamond” refers to a class structure in which a particular class appears more than once in a class’s inheritance hierarchy.
- Consider the situation where we have one class **A** .This class is **A** is inherited by two other classes **B** and **C**. Both these class are inherited into another in a new class **D** as shown in figure below.

diamond problem



- `class A{`
- `};`
- `class B : public A{`
- `};`
- `class C : public A{`
- `};`
- `class D : public B , public C{`
- `};`
- `int main(){`
- `cout << sizeof(A) << endl;`
- `cout << sizeof(B) << endl;`
- `cout << sizeof(C) << endl;`
- `cout << sizeof(D) << endl;`
- `}`

 C:\Users\Nida\Documents\Untitled1.exe

1
1
1
2

Process exited after 2.16 seconds with return value 0
Press any key to continue . . .

```
class A{
    int a;
};
class B : public A{
};
class C : public A{
};
class D : public B , public C{
};
int main(){
    cout << sizeof(A) << endl;
    cout << sizeof(B) << endl;
    cout << sizeof(C) << endl;
    cout << sizeof(D) << endl;
}
```

 C:\Users\Nida\Documents\Untitled1.exe

4
4
4
8

Process exited after 7.799 seconds with return value 0
Press any key to continue . . .

- `class A{`
- `public:`
- `int a;};`
- `class B : public A{`
- `public:`
- `int b;};`
- `class C : public A{`
- `public:`
- `int c;};`
- `class D : public B , public C{`
- `public:`
- `int d;};`
- `int main(){`
- `cout << sizeof(A) << endl;`
- `cout << sizeof(B) << endl;`
- `cout << sizeof(C) << endl;`
- `cout << sizeof(D) << endl;}`

 C:\Users\Nida\Documents\Untitled1.exe

4

8

8

20

Process exited after 7.704 seconds with return value 0

Press any key to continue . . .

diamond problem

- class A{
 - public:
 - int a;};
- class B : public A{
 - public:
 - int b;};
- class C : public A{
 - public:
 - int c;};
- class D : public B , public C{
 - public:
 - int d;};

diamond problem

- `int main(){`
- `D d;`
- `d.a;`
- `return 0;}`

This problem is also known as ***diamond problem***

- From above example we can see that “A” is inherited two times in D means an object of class “D” will contain two attributes of “a” (D::C::a and D::B::a).

Message

In function 'int main()':

[Error] request for member 'a' is ambiguous

[Note] candidates are: int A::a

[Note] int A::a

This problem is also known as ***diamond problem***

- As we can see from the figure that data members/function of class **A** are inherited twice to class **D**. One through class **B** and second through class **C**. When any data / function member of class **A** is accessed by an object of class **D**, ambiguity arises as to which data/function member would be called? One inherited through **B** or the other inherited through **C**. This confuses compiler and it displays error.

Memory layout of class “D” object:

Possible “D” object layout
A
B
A
C
D

Supposed memory layout of class “D” object

How to resolve this issue?

- To resolve this ambiguity when class **A** is inherited in both class **B** and class **C**, it is declared as **virtual base class** by placing a keyword **virtual** as :

How to resolve this issue?

1. There will be new virtual base pointers which would be inserted by compiler when we are using virtual inheritance.
2. This virtual base pointer will provide an indirect access to class "A" object
3. Identify the problematic class and use virtual keyword in all child classes that are associated with problematic class. In previous example problematic class is A that occurs more than once so inherit it by using virtual keyword

virtual base class / virtual inheritance

Syntax 1:

```
class B : virtual public A
{
};
```

Syntax 2:

```
class C : public virtual A
{
};
```

virtual base class / virtual inheritance

- Virtual base classes offer a way to save space and avoid ambiguities in class hierarchies that use multiple inheritances. When a base class is specified as a virtual base, it can act as an indirect base more than once without duplication of its data members. A single copy of its data members is shared by all the base classes that use virtual base.

virtual base class / virtual inheritance

- which single copy of its data members is shared?
- both base classes (class B and class C) use virtual base.

virtual base class / virtual inheritance

- it varies from compiler to compiler
- Depends on compiler which path is selected to access grand parents' data from grand child
- Usually, Last updated copy of its data members is shared by all the base classes that use virtual base
- The right most class being inherited is the one who's data member is going to come in derived class

virtual base class / virtual inheritance

```
class A {  
public:  
    int a;  
    A() {  
        a = 10;    }  
};  
class B : public virtual A {  
public:  
    B() {  
        a ++;    }  
};  
class C : public virtual A {  
public:  
    C() {  
        a ++;}};  
class D : public B ,public C {  
};
```

virtual base class / virtual inheritance

```
int main()
{
    D object; // object creation of class d
    cout << "a = " << object.a << endl;

    return 0;
}
```

virtual base
class / virtual
inheritance

```
D:\fall 2024\OneDrive\Docum  ×  +  ∨  
a = 12  
-----  
Process exited after 0.4464 seconds with return value 0  
Press any key to continue . . .
```

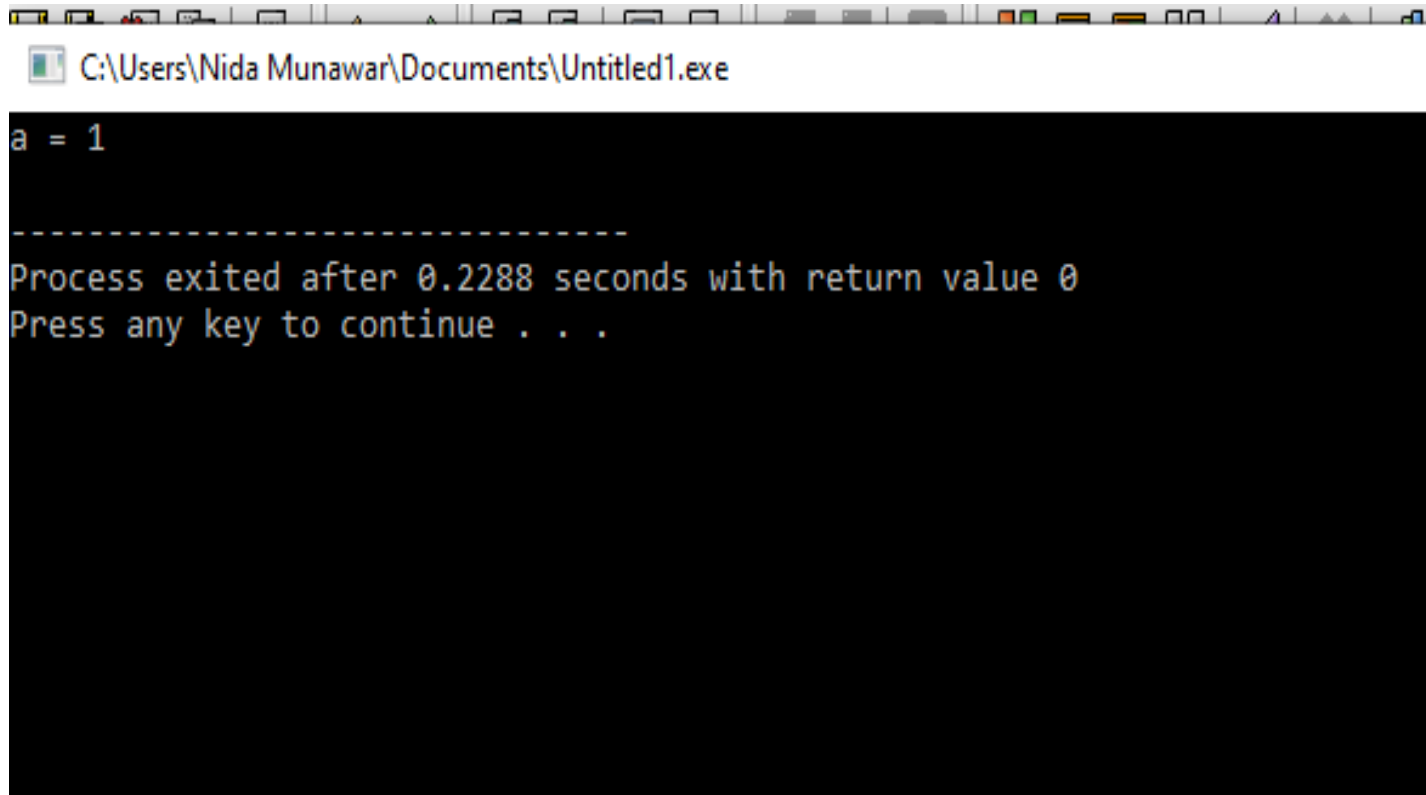
virtual base class / virtual inheritance

- class A {
- public:
- int a;
- A() {
- a = 10; }};
- class B : public virtual A {
- public:
- B() {
- a = 1; }};
- class C : public virtual A {
- public:
- C() {
- a = 2;}};
- class D : public C , public B {
- };

virtual base class / virtual inheritance

- `int main()`
- `{`
- `D object; // object creation of class d`
- `cout << "a = " << object.a << endl;`
- `return 0;`
- `}`

virtual base class / virtual inheritance



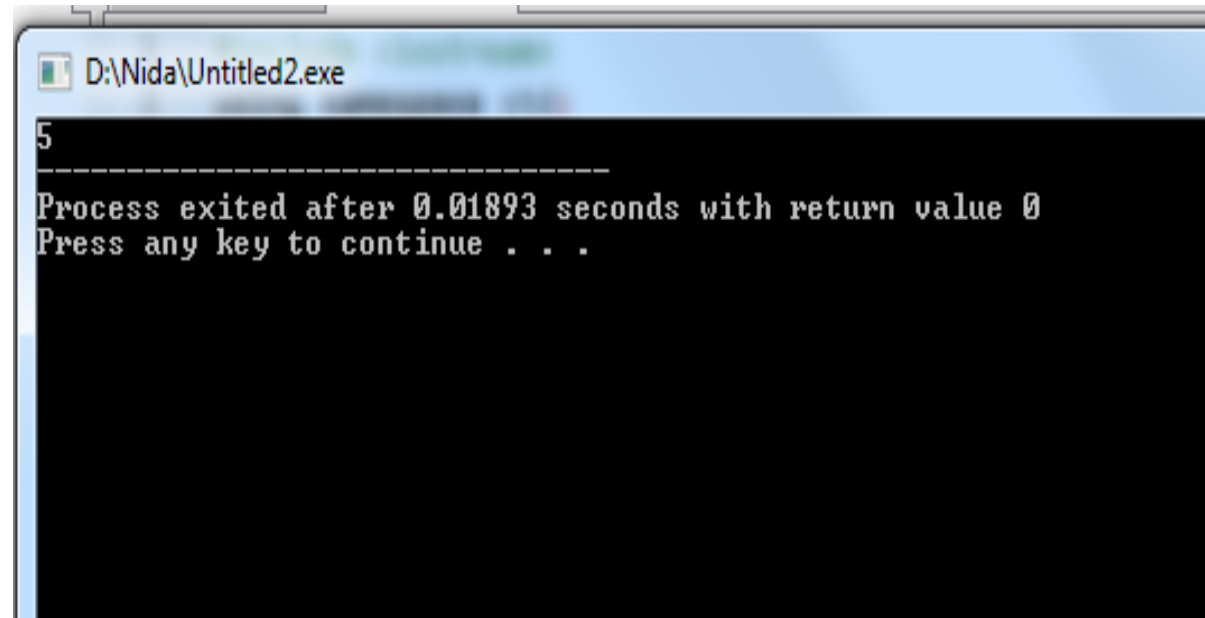
The screenshot shows a Windows command prompt window with the title bar "C:\Users\Nida Munawar\Documents\Untitled1.exe". The command prompt has a black background and white text. The first line of output is "a = 1". This is followed by a dashed line "-----". The next line of output is "Process exited after 0.2288 seconds with return value 0". The final line of output is "Press any key to continue . . .".

```
a = 1  
  
-----  
Process exited after 0.2288 seconds with return value 0  
Press any key to continue . . .
```

virtual base class / virtual inheritance

- `class D : public C , public B {`
- `public:`
- `D() {`
- `a = 5;} // now the last updated value will be 5`
- `getval(){`
- `cout << a;}};`

virtual base class / virtual inheritance



```
D:\Nida\Untitled2.exe
5
-----
Process exited after 0.01893 seconds with return value 0
Press any key to continue . . .
```

virtual base class / virtual inheritance

- `#include <iostream>`
- `using namespace std;`
- `class A {`
- `public:`
- `int a;`
- `A() {`
- `a = 10;}`
- `getval(){`
- `cout << a;`
- `};`

virtual base class / virtual inheritance

- class B : public virtual A {
- public:
- B() {
- a = 1;
- }
- getval(){
- cout << a;
- }};

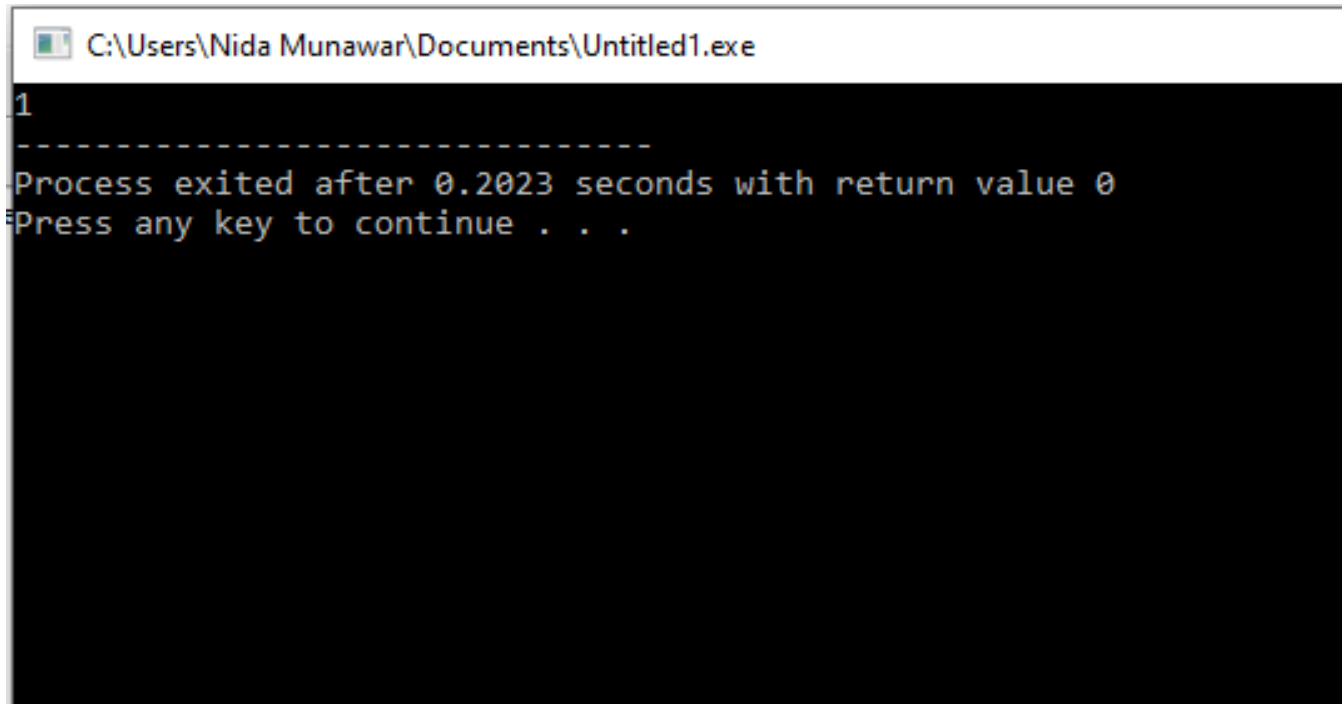
virtual base class / virtual inheritance

- class C : public virtual A {
 - public:
 - C() {
 - a = 2;}
 - getval(){
 - cout << a;}}
- class D : public C , public B {
 - public:
 - getval(){
 - C::getval();} // we are expecting it'll print 2

virtual base class / virtual inheritance

- `};`
- `int main()`
- `{`
- `D object; // object creation of class d`
- `object.getval();`
- `return 0;`
- `}`

Remember it will take rightmost inherited value of class B because we use virtual

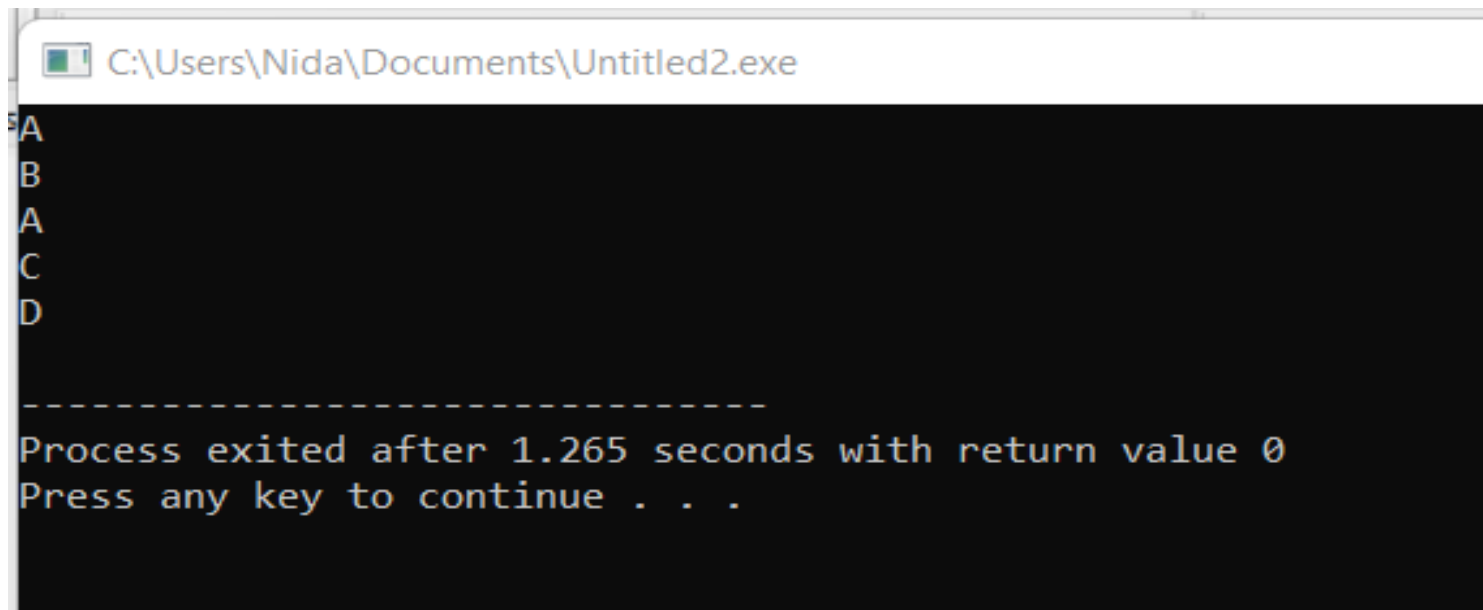


```
C:\Users\Nida Munawar\Documents\Untitled1.exe
1
-----
Process exited after 0.2023 seconds with return value 0
Press any key to continue . . .
```

Explanation

- You created object of class D compiler will choose one path from grand child to grand parent either from Class B or Class C
- In this case compiler will choose right most inherited class that is class B and print the value accordingly that's why it will hide the value updated in Class C
- If you want to access the updated value of class C you have to create the object of class C instead of class D it works normally

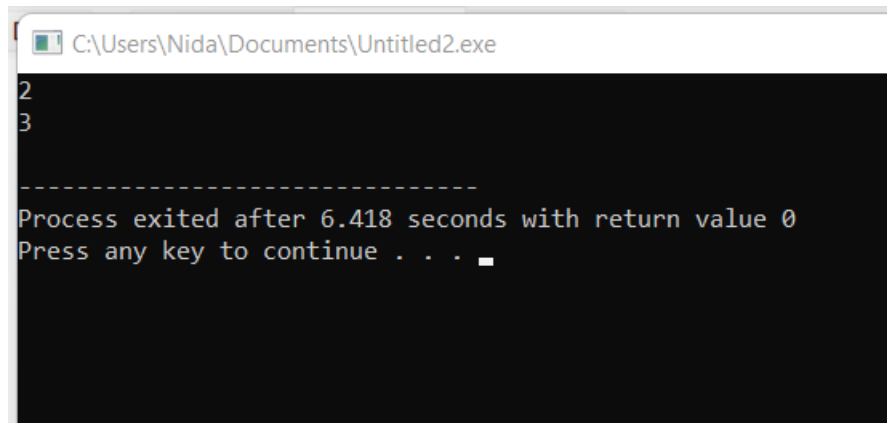
```
class A {  
public:  
    int a;  
    A() {  
        cout << "A" << endl;    };  
}  
  
class B : public A {  
public:  
    B() {cout << "B" << endl;    };  
}  
  
class C : public A {  
public:  
    C() {cout << "C" << endl;};  
}  
  
class D : public B ,public C {  
public:  
    D() {cout << "D" << endl;};  
}  
  
int main(){  
    D object; // object creation of class d  
    return 0;}
```



```
C:\Users\Nida\Documents\Untitled2.exe  
A  
B  
A  
C  
D  
  
-----  
Process exited after 1.265 seconds with return value 0  
Press any key to continue . . .
```

Without virtual

- class A {
- public:
- A(int a) {
- cout << a << endl; };
- class B : public A {
- public:
- B():A(2) { };
- class C : public A {
- public:
- C():A(3) {};
- class D : public B ,public C {
- public:
- D() { };
- int main(){
- D object; // object creation of class d
- return 0;}



```
C:\Users\Nida\Documents\Untitled2.exe
2
3
-----
Process exited after 6.418 seconds with return value 0
Press any key to continue . . .
```

With virtual

- class A {
- public:
- A(int a) {
- cout << a << endl; };
- class B : virtual public A {
- public:
- B():A(2) { };
- class C : virtual public A {
- public:
- C():A(3) {};
- class D : public B ,public C {
- public:
- D() { };
- int main(){
- D object; // object creation of class d
- return 0;}

 Compiler (7)  Resources  Compile Log  Debug  Find Results  Close			
Line	Col	File	Message
		C:\Users\Nida\Documents\Untitled2.cpp	In constructor 'D::D()':
16	6	C:\Users\Nida\Documents\Untitled2.cpp	[Error] no matching function for call to 'A::A()'
16	6	C:\Users\Nida\Documents\Untitled2.cpp	[Note] candidates are:
5	2	C:\Users\Nida\Documents\Untitled2.cpp	[Note] A::A(int)
5	2	C:\Users\Nida\Documents\Untitled2.cpp	[Note] candidate expects 1 argument, 0 provided
3	7	C:\Users\Nida\Documents\Untitled2.cpp	[Note] A::A(const A&)
3	7	C:\Users\Nida\Documents\Untitled2.cpp	[Note] candidate expects 1 argument, 0 provided

Now D is responsible to give value to its indirect parent class

- `#include <iostream>`
- `using namespace std;`
- `class A {`
- `public:`
- `A(int a) {`
- `cout << a << endl;   };`
- `class B : virtual public A {`
- `public:`
- `B():A(2)//compiler will ignore this but you cannot skip this line for hirarichal inheritance {                   };`
- `class C : virtual public A {`
- `public:`
- `C():A(3)//compiler will ignore this but you cannot skip this line for hirarichal inheritance {`
- `};`

- class D : public B ,public C {
- public:
- D():A(1) { }};
- int main()
- {
- D object; // object creation of class d
- return 0;
- }

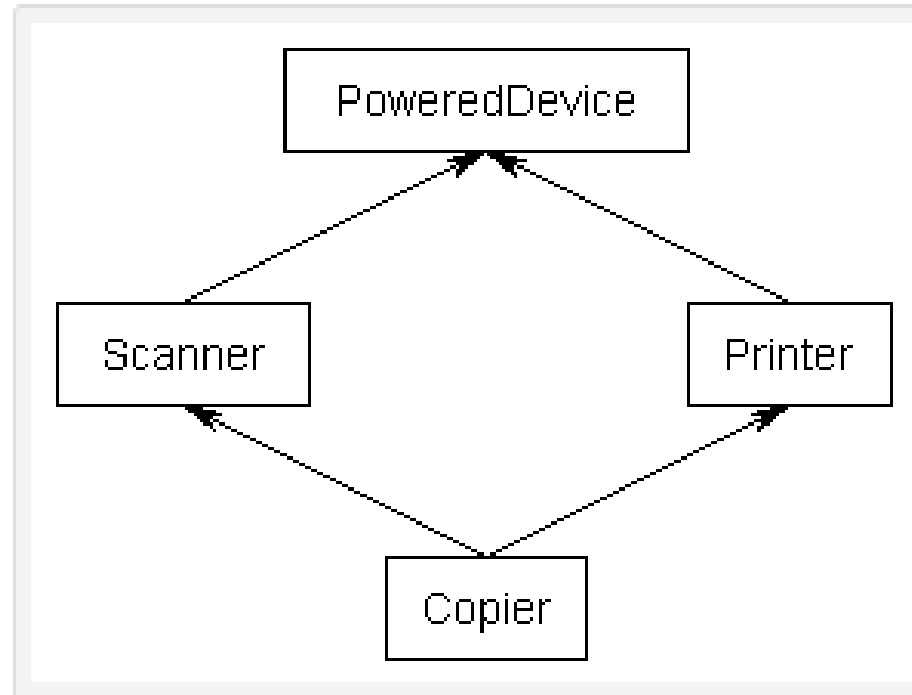
 C:\Users\Nida\Documents\Untitled2.exe

1

Process exited after 5.356 seconds with return value 0

Press any key to continue . . .

Identify the problematic class?



Identify the problematic class through parametrized constructor ?

- `class PoweredDevice{`
- `public:`
- `PoweredDevice(int power){`
- `cout << "PoweredDevice: " << power << "\n";}}`
- `class Scanner: public PoweredDevice{`
- `public:`
- `Scanner(int scanner, int power)`
- `: PoweredDevice(power) {`
- `cout << "Scanner: " << scanner << "\n"; }}`

Identify the problematic class through parametrized constructor ?

- `class Printer: public PoweredDevice{`
- `public:`
- `Printer(int printer, int power)`
- `: PoweredDevice(power) {`
- `cout << "Printer: " << printer << '\n'; }}}`
-
- `class Copier: public Scanner, public Printer{`
- `public:`
- `Copier(int scanner, int printer, int power)`
- `: Scanner(scanner, power), Printer(printer, power) {`
- `}};`

Identify the problematic class through parametrized constructor ?

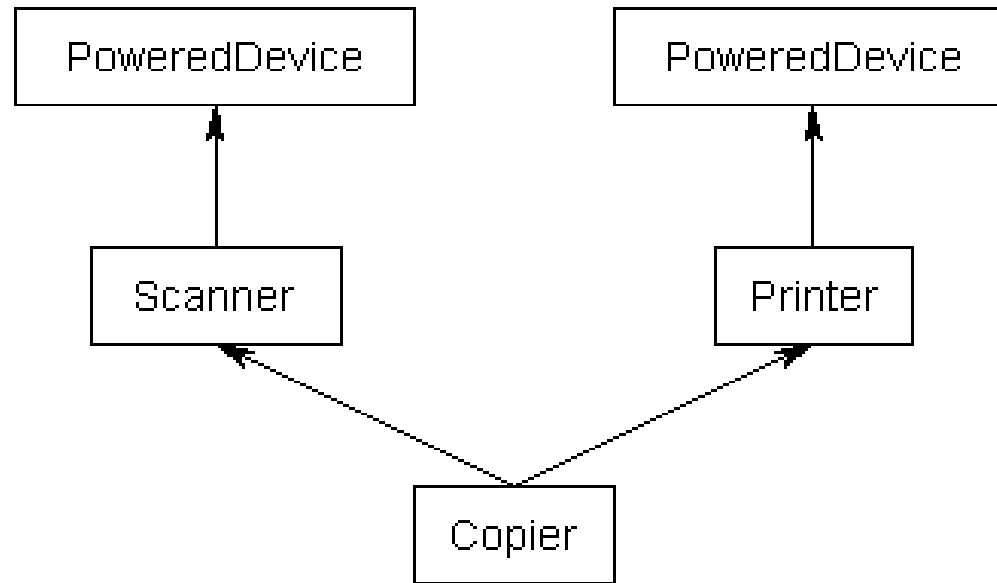
- `int main()`
- `{`
- `Copier copier(1, 2, 3);`
-
- `return 0;}`
- If you were to create a Copier class object, by default you would end up with two copies of the PoweredDevice class -- one from Printer, and one from Scanner. This has the following structure:

Identify the problematic class through parametrized constructor ?

```
C:\Users\Nida Munawar\Documents\param.exe
PoweredDevice: 3
Scanner: 1
PoweredDevice: 3
Printer: 2

-----
Process exited after 0.2705 seconds with return value 0
Press any key to continue . . .
```


Identify the problematic class through parametrized constructor ?



Solution By using Virtual Base class

- `#include <iostream>`
- `class PoweredDevice {`
- `public:`
- `PoweredDevice(int power) { std::cout << "PoweredDevice: " << power << '\n'; } }`
- `class Scanner: virtual public PoweredDevice`
- `{`
- `public:`
- `Scanner(int scanner, int power)`
- `: PoweredDevice(power) // this line is required to create Scanner objects, but ignored in this case because we are working with copier object`
- `{ std::cout << "Scanner: " << scanner << '\n'; } }`

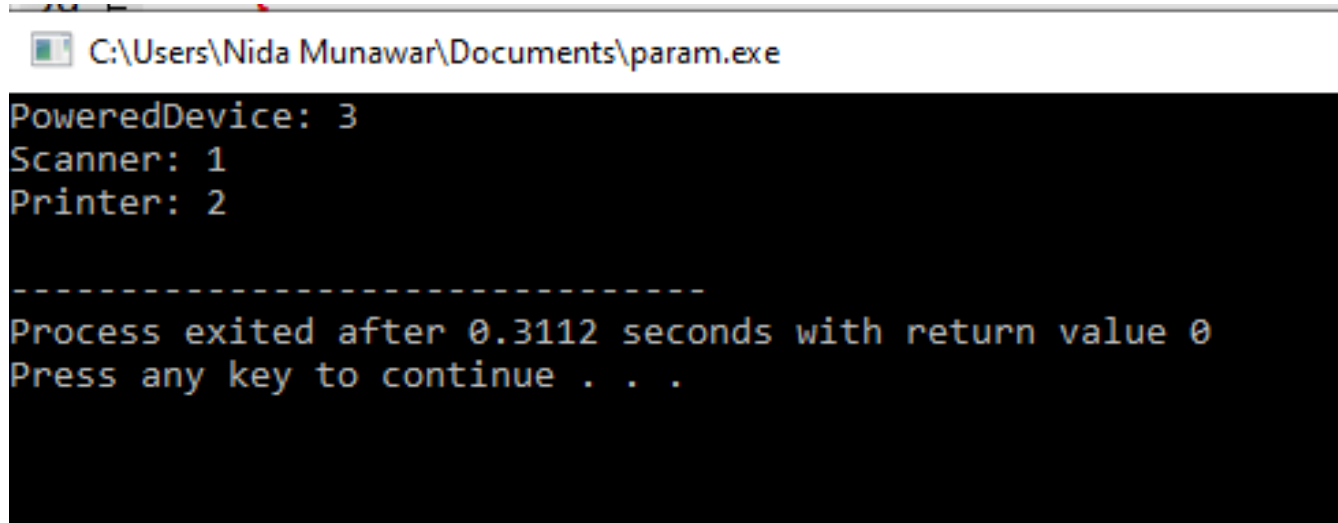
Solution By using Virtual Base class

- `class Printer: virtual public PoweredDevice // note: PoweredDevice is now a virtual base class`
- `{`
- `public:`
- `Printer(int printer, int power)`
- `: PoweredDevice(power) // this line is required to create Printer objects, but ignored in this case because we are working with copier object`
- `{ std::cout << "Printer: " << printer << '\n';`
- `};`

Solution By using Virtual Base class

- class Copier: public Scanner, public Printer
- {
- public:
- Copier(int scanner ,int printer, int power)
- : PoweredDevice(power), // PoweredDevice is constructed here updated value is provided by grandchild
- Scanner(scanner, power),Printer(printer, power)
- {
- }
- };
- int main()
- {
- Copier copier(1, 2, 3);
- return 0;}

Solution By using Virtual Base class



```
C:\Users\Nida Munawar\Documents\param.exe
PoweredDevice: 3
Scanner: 1
Printer: 2

-----
Process exited after 0.3112 seconds with return value 0
Press any key to continue . . .
```

Solution By using Virtual Base class

- Now, when you create a Copier class object, you will get only one copy of PoweredDevice per Copier that will be shared by both Scanner and Printer.
- However, this leads to one more problem: if Scanner and Printer share a PoweredDevice base class, who is responsible for creating it? The answer, as it turns out, is Copier. The Copier constructor is responsible for creating PoweredDevice. Consequently, this is one time when Copier is allowed to call a non-immediate-parent constructor directly if we not use virtual here we cannot access non-immediate-parent directly it will cause an compile time error